



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Introduzione a Javascript

Temi trasversali (Il parte)

Fabio Vitali

Corso di laurea in Informatica

Alma Mater – Università di Bologna

Oggi parleremo di...

Javascript

- Sintassi base (parte I)
- Javascript client-side (parte II)
- Sintassi avanzata (parte III)

Temi trasversali

- Navigazione sul DOM
- AJAX
- Programmazione asincrona
- ***Modularizzazione del codice***
- ***Interpolazione***
 - ***Template e interpolazione***
 - ***Componenti***
 - ***Framework a componenti***
- Routing
- Binding mono e bi-direzionale





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Modularizzazione in Javascript

Modularizzazione del codice

- Separazione del codice di un programma complesso in frammenti indipendenti e intercambiabili.
- Richiedono corretta decomposizione del problema, un'interfaccia alle funzioni chiara e precisa, e un buon incapsulamento delle funzionalità e delle strutture locali al modulo.
- Per lungo tempo Javascript non ha avuto un meccanismo di modularizzazione. Nessuna struttura sopra all'oggetto, nessuna possibilità di avere strutture private (fondamentali per l'incapsulamento).
- Poi, in rapida successione, arrivano prototipi, closure, IIFE, classi, moduli CommonJS e moduli EcmaScript.
- Inizialmente solo server-side, ora anche client-side



Aspetti comuni

- Separation of concern, incapsulamento, intercambiabilità del codice, load on demand.
- Ogni modulo è pensato in un file separato
- Le variabili possono essere:
 - Variabili locali alla funzione (come al solito) e
 - Variabili locali al modulo, globali per il modulo ma invisibili al di fuori del modulo
 - Variabili esportate dal modulo, visibili per chi include il modulo.
- Esistono modi per gestire la namespace pollution quando i moduli inclusi sono tanti



CommonJS e AMD

- Prima proposta di modularizzazione di Javascript (2009), adottata da NodeJS e da altri. NON FUNZIONA SUI BROWSER!
- Esiste una versione parzialmente compatibile, chiamata AMD (asynchronous module definition).
- Ogni modulo usa due costrutti speciali: `module.exports` e `require`:

```
// Saved as file "keys.js"
var keys = {
  8: 'backspace',
  9: 'tab',
  13: 'enter',
  16: 'shift',
  17: 'ctrl',
  18: 'alt'
};
var Utils = {
  name: function(keyCode) {
    return keys[keyCode];
  },
  keyCode: function(name) {
    return keys.indexOf(name);
  }
}
module.exports = Utils;
```

```
var keys = require('./keys.js');

document.addEventListener('keyup',
  function(e) {
    var key = keys.name(e.keyCode);
    console.log("You pressed "+key);
  }
);
```



ES module (1)

- La risposta a CommonJs da parte di WHATWG. A partire dal 2016, qualche anno dopo anche sui browser.
- Ogni modulo usa due istruzioni speciali: **export** e **import**:

```
// Saved as file "keys.js"
var keys = {
  8: 'backspace',
  9: 'tab',
  13: 'enter',
  16: 'shift',
  17: 'ctrl',
  18: 'alt'
};

export name: function(keyCode) {
  return keys[keyCode];
}
export keyCode: function(name) {
  return keys.indexOf(name);
}
```

```
import {name, keyCode} from ('../keys.js');

document.addEventListener('keyup',
  function(e) {
    var key = name(e.keyCode);
    console.log("You pressed "+key);
  }
);
```

oppure

```
import * as keys from ('../keys.js');

document.addEventListener('keyup',
  function(e) {
    var key = keys.name(e.keyCode);
    console.log("You pressed "+key);
  }
);
```

ES module (2)

- Se voglio esportare un solo nome (un oggetto, un variabile, una funzione, etc.) posso usare default export

```
// Saved as file "keys.js"
var keys = {
  8: 'backspace',
  9: 'tab',
  13: 'enter',
  16: 'shift',
  17: 'ctrl',
  18: 'alt'
};

name = function(keyCode) {
  return keys[keyCode];
}

export default name;
```

```
import newName from ('../keys.js');

document.addEventListener('keyup',
  function(e) {
    var key = newName(e.keyCode);
    console.log("You pressed "+key);
  }
);
```



ES module (3)

- Se voglio importare un modulo sul browser debbo esplicitamente indicare al browser che non sto caricando lo script totalmente (otterrei un errore), ma solo i nomi esplicitamente esportati dal modulo

Nell'HTML:

```
// Saved as file "keys.js"
var keys = {
  8: 'backspace',
  9: 'tab',
  13: 'enter',
  16: 'shift',
  17: 'ctrl',
  18: 'alt'
};

name = function(keyCode) {
  return keys[keyCode];
}

export default name;
```

```
<html>
  <head>
    <script type="module" src="keys.js"></script>
    <script>
      document.addEventListener('keyup',
        function(e) {
          var key = name(e.keyCode);
          console.log("You pressed "+key);
        }
      );
    </script>
  </head>
  ...
</html>
```



I moduli di node.js

- Node.js è estremamente modulare. E' possibile mettere codice indipendente in file javascript ed eseguirlo secondo necessità.
- Il meccanismo di caricamento di moduli di Node.js è semplice:
 - un modulo è un file Javascript.
 - Quando si richiede un modulo, esso viene cercato localmente o globalmente secondo un modello preciso.
- I moduli vengono caricati sia con `require()` sia con `import`.

```
http = require("http")
fs = require("redis")
require("./greetings.js")
console.log("You just loaded a lot of modules! ")
```

Modulo core built-in

dipendenza
(da installare con npm)

script locale





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Interpolazione

Interpolazione e template HTML

- I siti web dinamici creano le loro pagine HTML come risultato di un'operazione computazionale.
- Nei siti web LAMP più evoluti, l'HTML viene generato come ultima operazione dopo la identificazione di tutti i valori variabili risultato della logica applicativa (Architettura a quattro livelli).
- Spesso si utilizza per questo un template, ovvero una pagina priva di contenuti ma completa dal punto di vista grafico e tipografico, in cui vengono inseriti appositi elementi, detti placeholder, che verranno sostituiti con un valore calcolato dall'applicazione.
- I placeholder hanno una sintassi speciale, per distinguerli dal contenuto normale, e delle regole di composizione e arricchimento funzionale.



Interpolazione e template HTML: i problemi

- Ricorrere ad un linguaggio di programmazione per generare codice HTML però crea alcuni stupidi problemi sintattici solitamente ignorati dai linguaggi di programmazione tradizionali:
 - Stringhe molto lunghe e con ritorni a capo interni
 - Sovrapposizioni tra l'uso di alcuni caratteri (ad esempio, le virgolette) sia nel linguaggio di programmazione (delimitatori di stringa) sia nel linguaggio destinazione (delimitatori di valori di attributi)
 - Facilità di interpolazione (cioè di embedding di variabili della computazione in posizioni speciali del template)
- Non sono problemi "veri", ovvero non sono impediti queste funzioni da nessun linguaggio di programmazione.
- E' un problema di comodità e convenienza: alcuni linguaggi rendono queste cose più facili, veloci e meno prone ad errori.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Template e linguaggi di interpolazione

Interpolazione di variabili

soluzione 0: virgolette annidate

- Le virgolette sono caratteri ammessi in HTML, e sono caratteri fondamentali nel markup (ad esempio, attributi).
- HTML è esso stesso una stringa, che può essere prodotto programmaticamente. Le virgolette sono delimitatori standard delle stringhe.

```
var a ="<p class='"+obj.class+"'>" +obj.testo+"</p>"
```

- Se ho variabili da inserire nell'HTML, questo può portare a notevoli problemi di annidamento di stringhe in altre stringhe:

```
var b="<button onclick='doX(\""+obj.par+"\" ) '>" +obj.testo+"</button>"
```



Interpolazione di variabili

soluzione 1: funzioni di formattazione

Molti linguaggi di programmazione hanno funzioni e operatori per fare formattazione controllata di valori in una stringa in maniera posizionale:

- C/C++ :

```
int a = 1; int b = 2; char* v;  
sprintf(v, "La somma di %d e %d vale %d", a, b, a+b);
```
- Python:

```
a = 1  
b = 2  
v = "La somma di %d e %d vale %d" % (a, b, a+b) oppure  
v = "La somma di {0} e {1} vale {2}".format(a, b, a+b)
```

A volte ci sono anche meccanismi per formattazione di valori sulla base di nomi:

- Python:

```
v = "La somma di {x} e {y} vale {z}".format(x=a, y=b, z=a+b)
```
- PHP:

```
$a = 1; $b = 2; $c = $a+$b;  
$v = "La somma di $a e $b vale $c";
```

N.B.: in PHP l'interpolazione delle variabili è automatica nelle stringhe delimitate con doppi apici.



Interpolazione di variabili

soluzione 2: embedding di istruzioni

PHP nasce come motore di template (PHP significa *Hypertext Preprocessor*). Il meccanismo (poi esportato in altri linguaggi) di permettere embed di comandi eseguibili dentro ad una pagina HTML è un chiaro esempio di template.

```
<html>
  <head>
    <title><? echo $page_title;?></title>
  </head>
  <body>
    <h1><? echo $page_title;?></h1>
    <? echo $page_content; ?>
  </body>
</html>
```



Interpolazione di variabili

soluzione 3: heredoc

- La sostituzione delle variabili però non è sufficiente, perché nella maggior parte dei casi un documento HTML è molto lungo e diventa scomodissimo forzarlo su una riga soltanto.
- Gli here document (*heredoc*) sono sintassi (dipendenti da linguaggio a linguaggio) per avere valori stringa lunghi più di una riga nel programma.
- In PHP gli *heredoc* forniscono automaticamente interpolazione. In Python debbo usare la funzione `format`

Python

```
V = """\
<html>
<body>
    <h1>Hello {name}!</h1>
</body>
</html>
""".format(name="John")
```

PHP

```
$V = <<<EOS
<html>
<body>
    <h1>Hello $name!</h1>
</body>
</html>
EOS
```

Interpolazione di variabili

soluzione 4: template server-side

- I template server-side servono per generare un documento pronto a partire da un ambiente di computazione e un documento HTML vuoto con variabili da interpolare.
- Esistono anche per Javascript/Node, il più famoso è `express.render()`, spesso usato con Pug, che ha una sintassi semplificata per la parte HTML:

HTML (...circa: pug, in realtà)

```
doctype html
html(lang='en')
  head
    title #{title}
  body
    h1 #{title}
    div.container
      p #{message}
```

Node + Express.js

```
app.set('view engine', 'pug')
app.get('/', function (req, res){
  res.render('index.pug', {
    title: 'Ciao a tutti',
    message: 'Prima prova!'
  })
})
```



Interpolazione di variabili

Ok. Ma client-side?

Javascript non aveva né un meccanismo di interpolazione di variabili, né una sintassi per stringhe lunghe (*heredoc*).

Posso usare una stringa normale con i soliti problemi di lunghezza della riga e mancanza di interpolazione.

```
var str1 = "<p>prima riga di una stringa lunga,\n ";  
str1 += "seguita da una seconda riga,\n ";  
str1 += "e poi una terza riga.</p>";
```

Oppure posso bloccare la chiusura di una stringa con un backslash e proseguire sulla riga successiva:

```
var str1 = "<p>prima riga di una stringa lunga,\n \  
seguita da una seconda riga,\n \  
e poi una terza riga.</p>";
```



Interpolazione di variabili

soluzione 5: nascondere i template

- **Idea:** nascondere il testo del template nell'HTML stesso, ma inerte e inattivo, e poi quando serve prenderlo a inserirlo nel DOM in una parte visibile.
- Tuttavia debbo ricordarmi di proteggere il template HTML dalla sua visualizzazione, esecuzione, e/o indicizzazione prima del tempo o nel posto sbagliato.
- Per quanto riguarda il frammento, nel tempo si sono suggeriti vari meccanismi di protezione:
 - *div nascosti via CSS o posizionati fuori schermo.*
 - L'utente normale non lo vede, ma se il CSS viene disattivato lo vede. Inoltre un motore di ricerca lo indicizzerà lo stesso. Inoltre le immagini verranno comunque caricate e gli script eseguiti. Accessibilità da verificare. **PESSIMO!**
 - `<script type="text/template"> ... </script>`
 - Un truccaccio, ma funziona.



Interpolazione di variabili

soluzione 6: il tag `<template>`

Nell'HTML living standard è previsto il tag `<template>`.

Il contenuto di `<template>` è inerte e non fa parte del documento:

- Le immagini non vengono caricate,
- gli script non vengono eseguiti,
- Il contenuto non occupa spazio nel layout, ecc.

```
<template id="tpl">  
  <p>Testo</p>  
  <img src="" />  
</template>
```

Al momento utile, il template viene attivato importando il suo contenuto nel DOM come discendenti di un nodo del DOM esistente. In quel momento i contenuti si visualizzano, le immagini si caricano, gli script vengono eseguiti.

```
let tpl = document.getElementById('tpl').content ;  
let img = tpl.getElementsByTagName('IMG')[0] ;  
img.src = 'fig1.gif' ;  
let newContent = document.importNode(tpl) ;  
document.getElementById('output').appendChild(newContent) ;
```

Interpolazione di variabili

ok, ma la interpolazione?

- Adesso abbiamo i template, ma come facciamo per la sostituzione di variabili dentro ad un testo?
- Javascript ha la funzione `replace()`, ma è complicata e sostituisce una cosa alla volta:

```
let a = 1, b = 2, c = a+b;  
let v = "La somma di $primo e $secondo vale $terzo"  
    .replaceAll("$primo", a)  
    .replaceAll("$secondo", b)  
    .replaceAll("$terzo", a+b);
```

- JQuery non ha niente di accettabile.

C'è qualcosa di meglio?



Interpolazione di variabili

soluzione 7: interpolation dei poveri

```
String.prototype.tpl = function(o) {  
    var r = this ;  
    for (var i in o) {  
        r = r.replace(new RegExp("\\$" + i, 'g'), o[i])  
    }  
    return r  
}
```

Usarla con un array:

```
const t = "La somma di $0 e  
$1 vale $2";
```

```
let a=1, b=2;  
let v = t.tpl([a, b, a+b])
```

Usarla con un object:

```
var t = "<p>Io sono il $titolo  
$nome $cognome, ma potete  
chiamarmi $nome.</p>";
```

```
var o = {  
    titolo: "prof.",  
    nome: "Fabio",  
    cognome: "Vitali"  
} ;  
var r = t.tpl(o);
```


Interpolazione di variabili

soluzione 8: Template literals

Una sintassi per definire stringhe multi-linea con interpolazione di variabili.

```
var firstName = 'Jane';  
var x = `Hello ${firstName}!  
How are you  
today?`;
```

```
x vale "Hello Jane!  
How are you  
today?"
```

Tre elementi fondamentali: :

- Backticks come delimitatori: ``
- I new line fanno parte della stringa
- Interpolatori: `${varName}`

Attenzione! L'interpolazione avviene solo durante l'assegnazione del template literal, e non può essere controllata né posticipata.



Interpolazione di variabili

soluzione 9: tutto insieme

Se uso `<template>` e `string.prototype.tpl()`, posso ottenere risultati piacevoli in modo molto comodo:

```
<template id="tpl1">
  <p class="$classe">$testo</p>
</template>
<template id="tpl2">
  <button onclick='showDataFrom("$uri") '>$testo</button>
</template>
...
let data = {classe:"importante", testo:"p con testo vario"};

const out = document.getElementById('out');
const template1 = document.getElementById('tpl1').innerHTML;
const template2 = document.getElementById('tpl2').innerHTML;

out.innerHTML += template1.tpl(data);
out.innerHTML += template2.tpl({
  uri:"data.json",
  testo:"clicca qui per vedere i dati"
}));
```

Mustache.js

- Mustache è una semplice sintassi per template testuali con interpolazione di variabili
 - *logic-less*: nessun flusso di controllo, nessuna istruzione per cicli o condizioni;
 - *Separation of logic and presentation*: nessun modo per fare embedding di logica applicativa nel template.
- Mustache è disponibile su decine di linguaggi di programmazione; Mustache.js è la versione per Javascript.

```
Ciao {{name}} {{surname}}!  
Hai vinto {{value}} euro!  
{{#taxed}}  
  Beh, {{net}} euro, con le tasse.  
{{/taxed}}
```

```
Ciao Fabio Vitali!  
Hai vinto 10000 euro!  
Beh, 7800 euro, con le tasse.
```

```
{  
  "name": "Fabio",  
  "surname": "Vitali",  
  "value": 10000,  
  "net": 10000*0.78,  
  "taxed": true  
}
```



Handlebar.js

Handlebar.js è una semplice estensione di Mustache.js che permette di accedere agli oggetti annidati dentro alle variabili da interpolare e di ottenere l'effetto di **cicli** e **istruzioni condizionali** usando blocchi contestuali.

```
<div>
  <h1>{{title}}</h1>
  <h2>by
    {{#each authors}}
      <b>{{#if title}}{{title}}{{/if}}
        {{name}} {{surname}}
      </b>{{#unless @last}},{{/unless}}
    {{/each}}
  </h2>
  <div class="body">{{body}}</div>
</div>
```

```
{
  title: "My new post",
  authors: [{
    name: "John",
    surname: "Smith",
    title: "Dr."
  }, {
    name: "Mandy",
    surname: "Brown"
  }],
  body: "This is my first post!"
}
```

My new post

by Dr. John Smith, Mandy Brown

This is my first post!



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Interpolazione di variabili

soluzione 10: template client-side

In alternativa, posso ricorrere a framework appositi. Negli ultimi anni ne sono nati tre di grande successo e con una ragionevole somiglianza tra loro:

- Angular (poi evoluto in una vera e propria piattaforma applicativa con poca attinenza con il templating)
- ReactJS
- VueJS





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Componenti

Il limite dei linguaggi di markup

- Le applicazioni moderne stanno sempre più strette nel markup ufficialmente definito da WHATWG.
- Ci sono mille esigenze nuove che vengono raccolte lentamente e con molte cautele dal linguaggio.
- Un esempio per tutti i nuovi widget complessi:
 - menù di navigazione a scomparsa
 - pannelli modali
 - pannelli a tab
 - pannelli a fisarmonica (accordion)
 - Carousel
 - ... etc.
- Tutte queste funzionalità vengono gestite con strutture di markup + CSS + Javascript perfettamente funzionanti, ma molto lunghi e piuttosto complessi da leggere.
- Si veda Bootstrap, ma Tailwind non è meglio.



Esempi Bootstrap

```
// NAVBAR CON MENU A SCOMPARSA
<nav class="navbar navbar-expand-lg bg-body-tertiary">
  // TAB CON TRE PANNELLI
  <div class="container">
    // FINESTRA MODALE
    <div class="modal" tabindex="-1">
      <div class="modal-dialog">
        <div class="modal-content">
          <div class="modal-header">
            <h5 class="modal-title">Modal title</h5>
            <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
          </div>
          <div class="modal-body">
            <p>Modal body text goes here.</p>
          </div>
          <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
            <button type="button" class="btn btn-primary">Save changes</button>
          </div>
        </div>
      </div>
    </div>
    <div class="tab-pane fade show active" id="home-tab-pane" role="tabpanel" aria-labelledby="home-tab"
      tabindex="0">...</div>
    <div class="tab-pane fade" id="profile-tab-pane" role="tabpanel" aria-labelledby="profile-tab"
      tabindex="0">...</div>
    <div class="tab-pane fade" id="contact-tab-pane" role="tabpanel" aria-labelledby="contact-tab"
      tabindex="0">...</div>
  </div>
</nav>
```


Come sarebbe bello se...

```
// NAVBAR CON MENU A SCOMPARSA
```

```
<navbar>
  <brand>Navbar</brand>
  <menu id="homeButton">Home</menu>
  <menu id="linkButton">Link</menu>
  <menuBar title="Dropdown">
    <menu id="actionButton">Action</menu>
    <menu id="anotherActionButton">Another action</menu>
    <divider></divider>
    <menu id="somethingElseButton">Something else here</menu>
  </menubar>
  <menu id="actionButton" class="disabled">Disabled</menu>
</navbar>
```

```
// TAB CON TRE PANNELLI
```

```
<tab>
  <tabpanel title="Home" class="active">
    ...
  </tabpanel>
  <tabpanel title="Profile">
    ...
  </tabpanel>
  <tabpanel title="Contact">
    ...
  </tabpanel>
</tab>
```

```
// FINESTRA MODALE
```

```
<modal title="Modal title" id="modal1">
  <modalbody>
    <p>Modal body text goes here.</p>
  </modalbody>
  <modalfooter>
    <button id="closeB">Close</button>
    <button id="SaveB">Save changes</button>
  </modalfooter>
</modal>
```

XSLT

- Linguaggio di trasformazione da XML a XML o a HTML.
- Basato su template associati a XPath sul documento di input.

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="modal">
    <div class="modal" tabindex="-1" id="{@id}">
      <div class="modal-dialog">
        <div class="modal-content">
          <div class="modal-header">
            <h5 class="modal-title"><xsl:value-of select="@title"/></h5>
            <button type="button" class="btn-close" data-bs-dismiss="modal"
              aria-label="Close"></button>
          </div>
          <div class="modal-body">
            <xsl:apply-templates select="modalbody"/>
          </div>
          <div class="modal-footer">
            <xsl:apply-templates select="// FINESTRA MODALE
              <modal title="Modal title" id="modal1">
                <modalbody>
                  <p>Modal body text goes here.</p>
                </modalbody>
                <modalfooter>
                  <button id="closeB">Close</button>
                  <button id="SaveB">Save changes</button>
                </modalfooter>
              </modal>
            </div>
          </div>
        </div>
      </div>
    </div>
  </xsl:template>
  <xsl:template match="modalbody | modalbody" />
  <xsl:copy-of select="." />
</xsl:template>
</xsl:stylesheet>
```

Framework a componenti

- Un componente è un insieme anche complesso di codice dotato di senso compiuto, autonomo, autosufficiente e incapsulato.
- Il nuovo modello di web vede le applicazioni come costruzioni di molteplici componenti indipendenti che scambiano tra loro informazioni.
- Ogni componente si preoccupa di gestire
 - *la propria visualizzazione* (ad esempio, un frammento anche molto complesso di HTML e appositi frammenti di CSS)
 - *Il proprio modello di interazione con l'utente* (ad esempio, definendo eventi e callback in maniera appropriata)
 - *Il proprio meccanismo di interazione e scambio dati* con altri componenti e con l'ambiente generale dell'applicazione (ad esempio fornendo strutture dati e/o manipolando il markup e permettendo ad altri componenti di inserire la loro visualizzazione al proprio interno)



Termini comuni nei framework a componenti

- **Custom element:** estensioni di HTML (sia elementi sia attributi speciali) creati dalla libreria e/o dall'utente, che inseriscono in una posizione del DOM un componente
- **Template:** un frammento HTML che viene sostituito al custom element in fase di rendering (può contenere anche altri custom element, creando un processo ricorsivo di rendering)
- **Virtual DOM o Shadow DOM:** durante la fase di rendering il processo anche laborioso di navigazione ricorsiva tra template è realizzato su una copia nascosta del DOM, e non è percepito dall'utente, che vede qualcosa solo alla fine dell'intero processo di rendering.
- **Slot e interpolazione:** punti di collegamento tra componenti diversi in cui il template ospite fa spazio ai template di altri componenti (slot) o alla visualizzazione di dati che provengono dall'application logic.
- **Binding monodirezionale o bidirezionale:** un sistema di collegamento attivo tra dato dell'application logic e sua visualizzazione cosicché sia automatico l'aggiornamento della visualizzazione non appena viene calcolato un nuovo valore (monodirezionale) e viceversa quando l'utente modifica il valore attraverso widget di schermo (bidirezionale)



Lunga parentesi

- Le librerie a componente
 - Angular.js
 - Angular
 - React e Preact
 - Vue
- Node.js





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Dipartimento di Informatica – Scienze e Ingegneria
Alma mater – Università di Bologna

Fabio.vitali@unibo.it

www.unibo.it